

Stop Overthinking, Keep Passing the Tests: Shortest-Correct LoRA Fine-Tuning versus the Thinking Switch in a Small Code Model (Gemma-4-E4B)

Thesis proposal · September 2026 · 12 weeks · Budget: \$0 (free Colab and Kaggle GPUs)

1. Summary

Some new AI models "think" before they answer: they first write hidden notes, then the answer. On code problems this thinking is often much longer than needed, even for easy problems. Long thinking costs time and computing power. Many small 2026 models also have a free switch that turns thinking off. This thesis asks one question: **is it better to train a small model to think shorter, or to simply switch its thinking off?** We let the model solve each practice problem four times, keep its shortest answer that passes the tests, and train the model on these short answers. We then compare it with the free options on the same test problems. We expect it to use at least 25% less thinking while staying almost as accurate as normal thinking.

2. The Problem

A **large language model (LLM)** is a program that writes text by guessing the next word again and again. A **reasoning model** is an LLM that writes a thinking part before its answer. Thinking often makes answers more correct, but it is long. Length is counted in **tokens** (small pieces of text, about $\frac{3}{4}$ of a word). Every extra token costs time and money.

Research calls this **overthinking**: long thinking where short thinking would give the same answer. It is like a student who writes five pages for every exam question, even for " $2 + 2$ ".

There are two ways to fix it. The free way is to switch thinking off, or to cut it at a fixed length. The trained way is to teach the model to think shorter by itself. The free way costs nothing but may lose accuracy. The trained way costs some work. We do not yet know which one is better for code on a small model.

3. What Is Known and What Is Missing

Work	What it did	What it did not do
Self-Training Elicits Concise Reasoning (Munkhbat et al., 2025)	Kept the shortest correct answer and trained on it. Fewer tokens, similar accuracy.	Math only. No thinking switch.
SEER (2025)	Same idea on code tasks with a 7B model. About 40% shorter thinking.	Older model without a switch, so no comparison with thinking off.
ASAP (2025)	Removed unneeded steps from code thinking, then trained. 23.5% fewer tokens.	Older 7–8B models. No comparison with thinking off.
NoThinking (2025)	Showed that skipping thinking can be as good as short thinking, also on code.	No training. It shows that thinking off is a strong option we must beat.

The gap. We found no study that takes one small model with a thinking switch and compares, on code, a model trained to think shorter against the free options (thinking off, a length limit, or a "think briefly"

instruction).

What we do not claim. We are not the first to shorten thinking on code (SEER and ASAP did this). Our claim is only the fair, same-model comparison against the free options.

4. Research Question and Hypothesis

Research question. On a small model with a thinking switch, does training on its own shortest correct code answers give a better balance of accuracy and thinking length than the free options?

Hypothesis. Compared with normal thinking, the trained model:

- 1. uses at least **25% fewer** thinking tokens;
- 2. loses no more than **3 points** of accuracy;
- 3. is **more accurate** than thinking off, the length limit and the "think briefly" instruction.

The hypothesis is supported if all three parts hold on the test set. **It is contradicted** if a free option is as accurate as the trained model, or if accuracy drops by more than 3 points.

5. Method

- 1. **Sample.** The model answers each practice problem 4 times, with thinking on.
- 2. **Grade.** Each answer is run against the problem's real tests inside a **sandbox** (a separate, safe process with a time limit).
- 3. **Select.** For each problem, we keep the shortest answer that passes. To avoid teaching "always stop early", we skip answers shorter than half of the typical (median) correct length.
- 4. **Train.** We train a small **LoRA** add-on on these short correct answers. LoRA trains a small extra part and leaves the main model unchanged, so it fits on a free GPU.
- 5. **Test.** We compare five ways of answering on the same test problems: (1) thinking off, (2) thinking cut at a fixed length, (3) a "think briefly" instruction, (4) normal thinking, (5) normal thinking with our add-on.

6. Experiment Setup

Model. google/gemma-4-E4B-it (March 2026). It has a thinking switch, a free licence, and a free-GPU training notebook. Backup: Qwen/Qwen3.5-4B, used if Gemma does not fit in memory. We load a 4-bit version (a compressed copy, about 11 GB) so it fits on a free 15 GB T4 GPU.

Role	Data	Notes
Training	DeepCoder-Preview-Dataset (primeintellect and taco parts) and APPS introductory problems	Thousands of problems with tests. Examples kept under about 3,500 tokens.
Test	HumanEval+ (164), MBPP+ (378), LiveCodeBench easy and medium	About 1,000+ problems. Never used for training.

Scope. Easy and medium problems only. Hard problems need very long thinking, which does not fit on a free GPU, and a small model solves few of them.

Measures. Accuracy is **pass@1**: the share of problems solved on one try, averaged over 4 tries. Cost is the average number of thinking tokens. We report 95% confidence intervals with a paired **bootstrap** (re-sampling the same problems many times to see how much a difference could come from chance). One chart shows accuracy against thinking tokens for all five ways of answering.

Fairness rules. Every way of answering gets the same test problems, the same thinking limit (for example 8,000 tokens), the same prompts (except the "think briefly" one) and the same random seeds. Before training, we remove any practice problem that matches a test problem. All raw model outputs are saved, so results can be re-graded later.

To be checked in week 1: the model and dataset names, the LiveCodeBench problem counts, and whether the fast generation software (vLLM) runs Gemma-4 on a T4.

7. Two Checks Before the Big Runs

No one can promise in advance that training will help. But two cheap checks in week 3 tell us early whether it can. The limits are fixed now, so the data decides.

Check	Test	If it fails
Memory	Train on 50 examples. Peak GPU memory must be 14 GB or less.	Use the backup model, Qwen3.5-4B.
Headroom	The model answers 200 practice problems 4 times each. At least 40% of problems must have a correct answer, and the shortest correct answer must be at least 25% shorter than the average correct one.	Sample 8 answers per problem instead of 4, then check again.

8. Timeline

Weeks	Work	Output
1–2	Learn the tools. Set up Colab and Kaggle. Check the model and datasets. Build the sandbox grader. First model call with thinking on and off.	Working pipeline on a few problems
3	Run the memory and headroom checks.	Go / switch model / sample 8
4–5	Test the four ways of answering without training.	Baseline results
6–7	Build the training data: sample, grade, select. Remove overlap with the test set.	Training set of short correct answers
8–9	Train the LoRA add-on and test it.	Main comparison
10–12	Confidence intervals, chart, writing the thesis and paper.	Thesis, paper draft, released code

9. Risks

Risk	What we do
The model does not fit in free GPU memory	Memory check in week 3; use the backup model.
Short correct answers are not much shorter	Headroom check in week 3; sample 8 answers instead of 4.

Risk	What we do
Thinking off does as well as training	We report it honestly. It still answers the research question.
Free GPU sessions end without warning	Save results to Google Drive about every 20 problems.

10. Expected Contribution

This thesis gives one clear, fair answer: on a small 2026 model with a thinking switch, is training to think shorter worth it for code, compared with the free options? All work runs on free GPUs, and the code, raw outputs and trained add-on will be released so anyone can repeat it. If the hypothesis holds, the result shows a cheap way to make small models faster on code.

Future work (not part of this thesis): how much training data is needed, whether shorter thinking learned on code also works on math, and results on hard problems.

References

1. Sui et al. "Stop Overthinking: A Survey on Efficient Reasoning for Large Language Models." TMLR, 2025. arXiv:2503.16419.
2. Munkhbat et al. "Self-Training Elicits Concise Reasoning in Large Language Models." ACL Findings, 2025. arXiv:2502.20122.
3. SEER: adaptive chain-of-thought compression for software-engineering tasks. 2025. arXiv:2509.14093.
4. "Pruning the Unsurprising: Efficient Code Reasoning via First-Token Surprisal" (ASAP). 2025. arXiv:2508.05988.
5. "Reasoning Models Can Be Effective Without Thinking" (NoThinking). 2025. arXiv:2504.09858.
6. S3-CoT: self-sampled variable-length chain-of-thought. 2026. arXiv:2602.01982.
7. Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models." ICLR, 2022. arXiv:2106.09685.
8. Gemma 4 Technical Report. 2026. arXiv:2607.02770.
9. Liu et al. "Is Your Code Generated by ChatGPT Really Correct?" (HumanEval+, MBPP+). NeurIPS, 2023. arXiv:2305.01210.
10. Jain et al. "LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code." 2024. arXiv:2403.07974.
11. Miller. "Adding Error Bars to Evals: A Statistical Approach to Language Model Evaluations." 2024. arXiv:2411.00640.